Report on the Two-Stage Offset-Free Target Selector

March 24, 2026

1 Purpose

This report documents the target selector currently implemented in `Lyapunov/target_selector.py` through the function `compute_ss_target_refined_rawlings(...)`. The selector is designed for the standard Lyapunov tracking MPC path in this repository and computes a steady-state equilibrium target consistent with the current disturbance estimate.

The implementation follows a two-stage logic:

1. solve an exact offset-free steady-state target problem, and
2. if exact tracking is not admissible, solve a closest-feasible steady-state target problem.

This structure is motivated by the standard disturbance-augmented offset-free MPC formulation of Pannocchia and Rawlings [1], Pannocchia and Bemporad [2], and related tutorial and review material [3, 4, 5]. The closest-admissible fallback stage follows the reference tracking MPC idea of Limon et al. [6].

2 Context in the Repository

Within this project, the target selector is used upstream of the Lyapunov MPC tracking optimization. Its role is to convert a requested output setpoint into a consistent equilibrium triplet

$$(x_s, u_s, d_s), \tag{1}$$

where d_s is taken directly from the current disturbance estimate and is not re-optimized. The output of the target selector is then used to define the target-centered tracking problem and terminal ingredients in the Lyapunov MPC layer.

The code assumes the controller works in scaled deviation coordinates. This is important: the nominal input u_{nom} and the steady-state target variables are all interpreted in the same coordinate system as the MPC model. In the current codebase, this should normally mean that

$$u_{\text{nom}} \approx 0 \tag{2}$$

unless a different target-centering convention has been explicitly chosen.

3 Augmented Offset-Free Model

The starting point is the standard disturbance-augmented linear model used in offset-free MPC [1, 2]:

$$x_{k+1}^{\text{aug}} = A_{\text{aug}} x_k^{\text{aug}} + B_{\text{aug}} u_k, \tag{3}$$

$$y_k = C_{\text{aug}} x_k^{\text{aug}}, \tag{4}$$

with augmented state

$$x^{\text{aug}} = \begin{bmatrix} x \\ d \end{bmatrix}, \tag{5}$$

where $x \in \mathbb{R}^{n_x}$ is the physical state and $d \in \mathbb{R}^{n_d}$ is the disturbance state. In the current implementation, the augmented state ordering is assumed to be

$$x^{\text{aug}} = \begin{bmatrix} x \\ d \end{bmatrix}, \qquad n_d = n_y, \tag{6}$$

that is, the disturbance dimension is assumed equal to the output dimension.

The matrices are partitioned as

$$A_{\text{aug}} = \begin{bmatrix} A & B_d \\ 0 & I \end{bmatrix}, \tag{7}$$

$$B_{\text{aug}} = \begin{bmatrix} B \\ 0 \end{bmatrix}, \tag{8}$$

$$C_{\text{aug}} = \begin{bmatrix} C & C_d \end{bmatrix}. \tag{9}$$

Equivalently, the physical model is

$$x_{k+1} = A x_k + B u_k + B_d d_k, \tag{10}$$

$$d_{k+1} = d_k, \tag{11}$$

$$y_k = C x_k + C_d d_k. \tag{12}$$

Given the current augmented observer estimate

$$\hat{x}^{\text{aug}} = \begin{bmatrix} \hat{x} \\ \hat{d} \end{bmatrix}, \tag{13}$$

the target selector freezes the disturbance state at

$$d_s = \hat{d}. \tag{14}$$

This is the standard offset-free target-calculation logic: estimate the disturbance with the augmented observer, freeze the disturbance estimate, and compute the equilibrium state and input consistent with that estimate [2, 3, 4].

4 Steady-State Target Equations

At steady state, the physical state and input must satisfy

$$(I - A)x_s - Bu_s - B_d\hat{d} = 0. \tag{15}$$

The corresponding steady-state output is

$$y_s = Cx_s + C_d\hat{d}. \tag{16}$$

If all outputs are directly targeted, the controlled target expression is

$$y_c = y_s. \tag{17}$$

The implementation also supports an optional controlled-output selection matrix H, in which case

$$y_c = Hy_s, \tag{18}$$

so that the target vector y_{sp} lives in $\mathbb{R}^{n_c}$ rather than $\mathbb{R}^{n_y}$.

5 Constraints and Tightening

The selector supports input bounds

$$u_{\min} \leq u_s \leq u_{\max}, \tag{19}$$

and optional tightening:

$$u_\ell = u_{\min} + u_{\text{tight}}, \tag{20}$$
$$u_h = u_{\max} - u_{\text{tight}}. \tag{21}$$

These tightened input bounds are the actual bounds enforced in both target selection stages.

Optional output bounds may also be provided:

$$y_{\min} \leq y_s \leq y_{\max}. \tag{22}$$

When output tightening is enabled, the tightened bounds become

$$y_\ell = y_{\min} + y_{\text{tight}}, \tag{23}$$
$$y_h = y_{\max} - y_{\text{tight}}. \tag{24}$$

The implementation can handle the output bounds in two ways:

a) as hard inequalities, or

b) as soft inequalities with nonnegative slack variables.

The key design rule is that only the output inequalities may be softened. The exact target equality in the first stage is never softened.

6 Stage 1: Exact Offset-Free Steady-State Target

The first stage asks whether the requested setpoint is itself an admissible equilibrium for the disturbance-frozen model. The optimization problem is

$$\min_{x_s,u_s} \quad (u_s - u_{\text{nom}})^\top R_u (u_s - u_{\text{nom}}) + x_s^\top Q_x x_s \tag{25}$$

subject to

$$(I - A)x_s - Bu_s - B_d\hat{d} = 0, \tag{26}$$

$$y_c = y_{\text{sp}}, \tag{27}$$

$$u_\ell \le u_s \le u_h. \tag{28}$$

If hard output bounds are enabled, they are also enforced:

$$y_\ell \le y_s \le y_h. \tag{29}$$

If soft output bounds are enabled, the code instead introduces $s_\ell, s_h \in \mathbb{R}^{n_y}_{\ge 0}$ and uses

$$y_s + s_\ell \ge y_\ell, \tag{30}$$

$$y_s - s_h \le y_h, \tag{31}$$

while augmenting the cost by

$$s_\ell^\top W_\ell s_\ell + s_h^\top W_h s_h. \tag{32}$$

The matrix Q_x is not meant to bias the target significantly. It acts as a small regularizer to make the state solution numerically well posed when multiple equilibria correspond to similar inputs. The dominant Stage 1 design principle is exact target satisfaction, not compromise between target error and control effort.

7 Stage 2: Closest-Admissible Fallback Target

If Stage 1 is infeasible or numerically unacceptable, the implementation falls back to a closest-feasible equilibrium problem. This is the correct place to penalize target mismatch, because at this point exact tracking has already been ruled out. The fallback problem is

$$\min_{x_s,u_s} \quad (y_c - y_{\text{sp}})^\top T_y (y_c - y_{\text{sp}}) + (u_s - u_{\text{nom}})^\top R_u (u_s - u_{\text{nom}}) + x_s^\top Q_x x_s. \tag{33}$$

If previous target information is provided, the code adds smoothing terms

$$\begin{aligned}(x_s - x_s^{\text{prev}})^\top Q_{dx} (x_s - x_s^{\text{prev}}) \\ + (u_s - u_s^{\text{prev}})^\top R_{du} (u_s - u_s^{\text{prev}}).\end{aligned} \tag{34}$$

These regularizers are only used in the fallback stage. They are intentionally excluded from Stage 1 so that exact target satisfaction is never weakened by target-motion smoothing.

The fallback constraints are

$$(I - A)x_s - Bu_s - B_d\hat{d} = 0, \tag{35}$$

$$u_\ell \le u_s \le u_h, \tag{36}$$

plus the same hard or soft output-bound structure, if output bounds are present. There is no hard equality $y_c = y_{\text{sp}}$ in Stage 2.

This formulation is aligned with the closest-admissible steady-state idea used in constrained tracking MPC [6]: if the requested target cannot be exactly achieved, the controller should move toward the nearest admissible steady state rather than forcing an infeasible reference.

8 Numerical Acceptance Logic

The code does not accept a target solely because a CVXPY solver returns `optimal` or `optimal_inaccurate`. Instead, it validates the solution using explicit residual checks.

8.1 Dynamic Residual

For either stage, the dynamic residual is

$$r_{\text{dyn}} = (I - A)x_s - Bu_s - B_d\hat{d}. \tag{37}$$

The implementation computes

$$\|r_{\text{dyn}}\|_\infty . \tag{38}$$

8.2 Target Equality Residual

For Stage 1, the exact target residual is

$$r_{\text{eq}} = y_c - y_{\text{sp}}, \tag{39}$$

with infinity norm

$$\|r_{\text{eq}}\|_\infty . \tag{40}$$

8.3 Tolerance Policy

The implementation uses the following acceptance thresholds:

$$\|r_{\text{dyn}}\|_\infty \le 10^{-6} \qquad \text{if the status is } \texttt{optimal}, \tag{41}$$

$$\|r_{\text{dyn}}\|_\infty \le 10^{-5} \qquad \text{if the status is } \texttt{optimal_inaccurate}. \tag{42}$$

For Stage 1 the same threshold policy is applied to

$$\|r_{\text{eq}}\|_\infty . \tag{43}$$

If hard output bounds are active, the implementation also checks the numerical bound violation magnitude and rejects the candidate if the hard constraints are violated beyond tolerance.

This acceptance discipline is important in practice, particularly when fallback solves or soft-bound formulations are passed to first-order or inexact solvers. The target selector must return a true equilibrium, not just a nominally solved optimization variable tuple.

9 Returned Values and Debug Information

On success, the function returns

$$(x_s, u_s, d_s), \qquad d_s = \hat{d}. \tag{44}$$

It also constructs the augmented target

$$x_s^{\text{aug}} = \begin{bmatrix} x_s \\ \hat{d} \end{bmatrix}, \tag{45}$$

which is useful for the Lyapunov tracking formulation.

The debug dictionary is intentionally richer than the basic return tuple. It includes, among other fields:

- `solve_stage`, indicating whether the accepted solution came from the exact or fallback stage,
- `target_error`, `target_error_inf`, and `target_error_norm`,
- `target_eq_residual_inf`,
- `dyn_residual_inf`,
- `bound_violation_inf`,
- `x_s_aug`,
- soft-bound slack values when applicable,
- stage-specific status and reject-reason information.

For backward compatibility with existing diagnostics, the implementation also defines

$$\texttt{target_slack} := \texttt{target_error}, \tag{46}$$

$$\texttt{target_slack_inf} := \texttt{target_error_inf}. \tag{47}$$

These names are retained even though the new implementation no longer uses a single explicit output-tracking slack variable in the main exact-target solve.

10 Why the Two-Stage Structure Matters

The main conceptual reason for the redesign is that the target selector should not blend exact offset-free equilibrium calculation and infeasibility handling into one weighted compromise problem. Those are two distinct tasks:

1. If the requested setpoint is feasible for the disturbance-frozen model, the target selector should return the exact equilibrium corresponding to that setpoint.
2. Only if the setpoint is not feasible should the target selector solve a nearest-admissible target problem.

This distinction is especially important in the present repository because the Lyapunov tracking MPC uses the selected steady-state target as the center for tracking errors, terminal costs, and terminal-set reasoning. If the target selector prematurely softens the target even when exact tracking is feasible, then the Lyapunov layer is centered around an artificially biased equilibrium.

11 Mapping Between Code Arguments and Mathematical Objects

`Python argument or field`	Mathematical role
`A_aug, B_aug, C_aug`	Augmented offset-free prediction model
`xhat_aug`	Current augmented estimate $[\hat{x}; \hat{d}]$
`y_sp`	Requested steady-state target y_{sp}
`u_min, u_max`	Untightened input limits
`u_tight, y_tight`	Tightening vectors for target admissibility
`u_nom`	Input-centering reference u_{nom}
`Ty_diag`	Diagonal of the fallback target-mismatch weight T_y
`Ru_diag`	Diagonal of the input-centering weight R_u
`Qx_diag`	Diagonal of the state regularization matrix Q_x
`Qdx_diag, Rdu_diag`	Previous-target regularization weights Q_{dx}, R_{du}
`y_min, y_max`	Optional output constraints
`Wy_low_diag, Wy_high_diag`	Soft output-bound penalties W_ℓ, W_h
`H`	Optional output-selection matrix for controlled outputs
`solve_stage`	Indicator of whether Stage 1 or Stage 2 produced the accepted target

12 Recommended Method Description for a Thesis or Paper

A concise method description consistent with the implemented selector is:

> The steady-state target selector is based on a standard offset-free MPC formulation using a disturbance-augmented linear prediction model and observer-based disturbance estimation. At each control step, the current disturbance estimate is frozen and an exact steady-state target problem is solved for the equilibrium state and input consistent with the requested setpoint. If the requested setpoint is infeasible under the disturbance-frozen model and the imposed constraints, the selector computes the closest admissible steady-state target instead. This yields a target calculation that is exact when feasible and only softened when infeasibility makes exact tracking impossible.

13 Compilation Notes

This report is written as a self-contained LaTeX file with an internal bibliography, so it can be compiled with plain `pdflatex` without `bibtex` or `biber`. On a machine with a LaTeX installation, run:

```
cd report
pdflatex -interaction=nonstopmode target_selector_report.tex
pdflatex -interaction=nonstopmode target_selector_report.tex
```

The second pass resolves cross-references and citation numbering cleanly.

References

[1] G. Pannocchia and J. B. Rawlings, "Disturbance models for offset-free model-predictive control," *AIChE Journal*, 2003.

[2] G. Pannocchia and A. Bemporad, "Combined design of disturbance model and observer for offset-free model predictive control," *IEEE Transactions on Automatic Control*, 2007.

[3] U. Maeder and M. Morari, "Offset-free reference tracking with model predictive control," *Automatica*, 2010.

[4] G. Pannocchia, "Offset-free tracking MPC: A tutorial review and comparison of different formulations," in *Proceedings of the European Control Conference*, 2015.

[5] G. Pannocchia, M. Gabiccini, and A. Artoni, "Offset-free MPC explained: novelties, subtleties, and applications," *IFAC-PapersOnLine*, 2015.

[6] D. Limon, I. Alvarado, T. Alamo, and E. F. Camacho, "MPC for tracking piecewise constant references for constrained linear systems," *Automatica*, 2008.